Sztuczna Inteligencja w Grafice Komputerowej

Projekt 1: Modyfikacja obrazów

Zwiększanie rozdzielczości i odszumianie obrazów siecią rezydualną ResidualNet

Sprawozdanie z realizacji projektu

Zespół: Kacper Preyzner
Rafał Zwierzyński

Wybrane zadania: 2.1 Zwiększanie rozdzielczości, 2.2 Odszumianie

W skrócie. Zrobiliśmy dwa z czterech zadań: zwiększanie rozdzielczości ($64\times64 \to 256\times256$ oraz $32\times32 \to 256\times256$) i odszumianie (szum gaussowski $\sigma = 0{,}01$ i $0{,}03$). Do obu zadań użyliśmy tej samej, małej sieci rezydualnej, która uczy się *poprawki* do obrazu wejściowego. Na 100 obrazach testowych sieć była lepsza od metod bazowych (bicubic z OpenCV i `denoise_bilateral` ze scikit-image) w każdym z czterech wariantów i w każdej metryce: SNE, PSNR, SSIM i LPIPS. Przy odszumianiu zysk PSNR wyniósł +3,8 i +4,7 dB, a przy zwiększaniu rozdzielczości +0,7 i +0,5 dB.

Spis treści

1 Cel projektu i wybór zadań **1**

2 Dane i ich przygotowanie **2**
2.1 Zbiór DIV2K i podział . . . 2
2.2 Zwiększanie rozdzielczości: przygotowanie wejść . . . 2
2.3 Odszumianie: przygotowanie wejść . . . 3

3 Metody bazowe **3**

4 Proponowana architektura: ResidualNet **3**

5 Trening **4**
5.1 Eksperyment: FP32 czy BF16? . . . 5

6 Metryki **6**

7 Wyniki **6**
7.1 Zwiększanie rozdzielczości . . . 6
7.2 Odszumianie . . . 7
7.3 Wyniki dla pojedynczych obrazów . . . 7
7.4 Wyniki wizualne . . . 7

8 Wnioski **9**

9 Ograniczenia i co można by zrobić dalej **9**

10 Jak uruchomić **10**

A Przykłady z raportu dla obrazów testowych **10**

1 Cel projektu i wybór zadań

Zgodnie z instrukcją mieliśmy zaproponować architekturę sieci neuronowej, wykorzystać ją do dwóch z czterech zadań modyfikacji obrazów i porównać wyniki z metodami bazowymi przy pomocy metryk z wykładu. Wybraliśmy:

- **zwiększanie rozdzielczości** (super-resolution, SR) z obrazów 64×64 (powiększenie $\times 4$) i 32×32 (powiększenie $\times 8$) do 256×256,
- **odszumianie** obrazów z addytywnym szumem gaussowskim o $\sigma = 0{,}01$ i $\sigma = 0{,}03$.

Oba zadania pasują do wspólnego modelu degradacji z wykładu W04, $y = D(x) + n$: przy SR operacja D to zmniejszenie rozdzielczości, a przy odszumianiu D jest identycznością i zostaje tylko szum n. Dlatego mogliśmy użyć jednej rodziny sieci, zmieniając tylko końcowy moduł powiększania. Dla każdego z czterech wariantów (`sr4`, `sr8`, `dn001`, `dn003`) wytrenowaliśmy osobny model.

Tabela 1 zestawia wymagania z instrukcji z miejscami, w których opisujemy ich realizację.

Tabela 1. Wymagania z instrukcji i ich realizacja.

Wymaganie	Realizacja	Sekcja
Python + PyTorch	cały kod w pakiecie `sigk`, sieć i trening w PyTorch	4, 5
Własna architektura	ResidualNet: sieć rezydualna w stylu EDSR	4
2 z 4 zadań	zwiększanie rozdzielczości i odszumianie, po dwa warianty	2
DIV2K, obrazy 256 × 256	wszystkie obrazy przeskalowane do 256 × 256	2
Metoda zmiany rozdzielczości i opis interpolacji	`cv2.resize`, INTER_AREA do zmniejszania	2.2
Metoda zaszumiania	szum $\mathcal{N}(0, \sigma^2)$ na obrazie w $[0, 1]$	2.3
Metody bazowe	bicubic (OpenCV), `denoise_bilateral` (scikit-image)	3
Metryki SNE, PSNR, SSIM, LPIPS	wszystkie cztery, liczone dla każdego obrazu	6
Wyniki na całym zbiorze testowym	wszystkie 100 obrazów testowych w każdym wariancie	7
Tabela dla każdego zadania	tabele 8 i 9	7
Wyniki wizualne i wnioski	rysunki 4–6, wnioski	7.4, 8
Lepszy wynik niż metoda bazowa	spełnione we wszystkich wariantach i metrykach	7

2 Dane i ich przygotowanie

2.1 Zbiór DIV2K i podział

Korzystaliśmy z publicznych archiwów DIV2K: 800 obrazów treningowych w wysokiej rozdzielczości (0001–0800) i 100 obrazów walidacyjnych (0801–0900). Obrazy testowe DIV2K (0901–1000) nie mają publicznie udostępnionych wersji HR, więc nie da się na nich policzyć metryk porównujących z oryginałem. Dlatego jako **zbiór testowy** wykorzystaliśmy oficjalny zbiór walidacyjny DIV2K, a własny zbiór walidacyjny (do wyboru epoki i parametrów filtru) wydzieliliśmy z części treningowej (tabela 2). Zbiór testowy nie był używany do żadnych decyzji – tylko do końcowego pomiaru.

Tabela 2. Podział danych. Podziały są rozłączne (sprawdzane po sumach SHA-256 plików źródłowych).

Podział	Obrazy DIV2K	Do czego służy	Liczba
trening	0001–0720 (train HR)	uczenie wag sieci	720
walidacja	0721–0800 (train HR)	wybór epoki, strojenie filtru bilateralnego	80
test	0801–0900 (valid HR)	końcowe wyniki we wszystkich tabelach	100

Każdy obraz został w całości przeskalowany do 256 × 256 funkcją `cv2.resize` z interpolacją INTER_AREA (bez wycinania fragmentów). Oryginały DIV2K nie są kwadratowe, więc obrazy są lekko ściśnięte w poziomie. Przyjęliśmy to dla prostoty – wszystkie metody dostają dokładnie te same obrazy, więc porównanie pozostaje uczciwe. Przygotowane pliki mają zapisane sumy SHA-256 w manifeście (`docs/DIV2K_manifest.json`), a trening i ewaluacja sprawdzają, czy dane się nie zmieniły.

2.2 Zwiększanie rozdzielczości: przygotowanie wejść

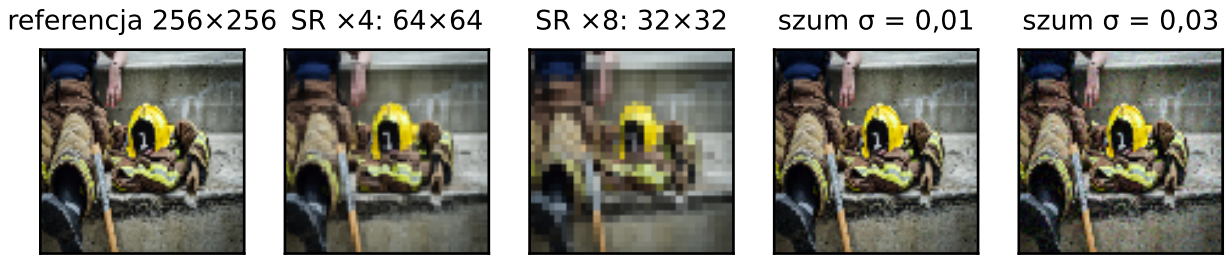
Wejścia o niskiej rozdzielczości tworzymy z obrazów 256 × 256 funkcją `cv2.resize(obraz, (64, 64))` lub `(32, 32)` z interpolacją INTER_AREA. Ta metoda uśrednia piksele z obszaru, który trafia do jednego piksela wyniku, czyli działa jak filtr dolnoprzepustowy. Na wykładzie W04 było pokazane, że filtrowanie przed zmniejszeniem rozdzielczości ogranicza aliasing – dlatego nie użyliśmy INTER_NEAREST ani INTER_LINEAR. Do powiększania z powrotem do 256 × 256 (metoda bazowa i wejście pomocnicze naszej sieci) używamy INTER_CUBIC, zgodnie z instrukcją.

2.3 Odszumianie: przygotowanie wejść

Obraz w zakresie $[0{,}0;\ 1{,}0]$ (typ `float32`) zaszumiamy wzorem

$$y = \mathrm{clip}\big(x + n,\ 0,\ 1\big), \qquad n \sim \mathcal{N}(0, \sigma^2), \quad \sigma \in \{0{,}01;\ 0{,}03\}, \tag{1}$$

gdzie szum jest niezależny dla każdego piksela i kanału RGB, a σ to odchylenie standardowe. Używamy `numpy.random.Generator.normal`, co daje to samo co `skimage.util.random_noise(x, mode="gaussian", var=`σ^2`)` – ta funkcja przyjmuje *wariancję*, a nie σ, więc łatwo tu o pomyłkę. Szum na zbiorach walidacyjnym i testowym jest deterministyczny: ziarno losowania wyliczamy z nazwy pliku i wariantu, więc każda metoda dostaje identyczny zaszumiony obraz. W treningu szum losujemy od nowa w każdej epoce.



Rysunek 1. Przykładowe wejścia dla czterech wariantów (obraz 0721 z naszej walidacji). Wejścia SR są pokazane powiększone metodą nearest, żeby było widać prawdziwą liczbę pikseli. Szum $\sigma = 0{,}01$ jest prawie niewidoczny, przy $\sigma = 0{,}03$ widać już ziarno na jednolitych powierzchniach.

3 Metody bazowe

Zwiększanie rozdzielczości. Powiększenie wejścia do 256×256 funkcją `cv2.resize` z parametrem `interpolation=cv2.INTER_CUBIC`; wynik przycinamy do $[0, 1]$.

Odszumianie. Funkcja `denoise_bilateral` z modułu `skimage.restoration` z parametrami `channel_axis=-1` i `mode="reflect"`. Instrukcja nie podaje parametrów filtru, a domyślne `sigma_color` w scikit-image jest wyliczane z całego obrazu, co dla tak słabego szumu daje mocne rozmycie. Żeby porównanie było uczciwe, dobraliśmy parametry na naszym zbiorze walidacyjnym (80 obrazów), ustawiając `sigma_color` $= k \cdot \sigma$ i sprawdzając $k \in \{1, 2, 3\}$ oraz `sigma_spatial` $\in \{1, 3\}$ (tabela 3). W obu wariantach wygrało $k = 1$, `sigma_spatial` $= 3$ i tych wartości używamy w teście.

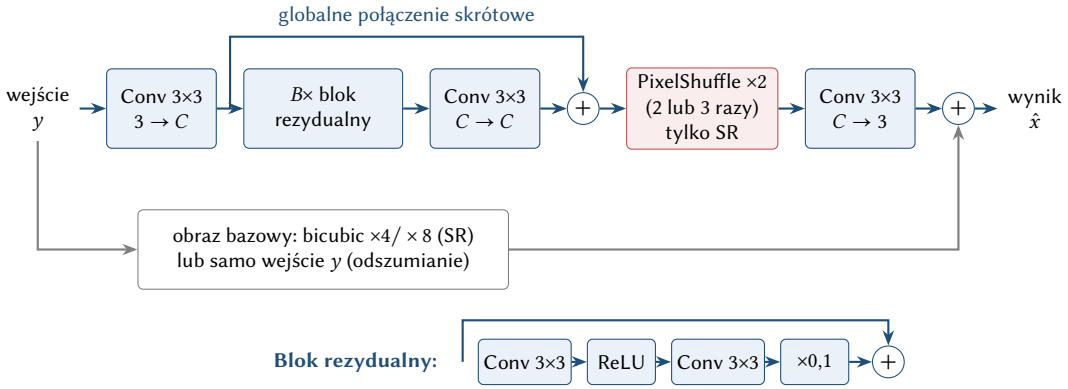
Tabela 3. Strojenie filtru bilateralnego: średni PSNR [dB] na 80 obrazach walidacyjnych.

	$\sigma = 0{,}01$			$\sigma = 0{,}03$		
`sigma_color`	1σ	2σ	3σ	1σ	2σ	3σ
`sigma_spatial` = 1	36,48	32,87	30,58	30,10	26,54	24,48
`sigma_spatial` = 3	**38.93**	34,80	32,01	**31.51**	27,19	24,64

Strojenie dużo zmienia – dla $\sigma = 0{,}03$ różnica między najlepszym a najgorszym ustawieniem to ok. 7 dB. Najlepsze $k = 1$ leży na brzegu sprawdzanej siatki, więc możliwe, że jeszcze mniejsze `sigma_color` dałoby trochę lepszy wynik. Tego już nie sprawdzaliśmy.

4 Proponowana architektura: ResidualNet

Naszą sieć oparliśmy na pomysłach z wykładu W04, głównie z EDSR, VDSR i DnCNN. Schemat pokazuje rysunek 2.



Rysunek 2. Architektura ResidualNet. Na niebiesko część wspólna dla obu zadań, na czerwono moduł powiększania używany tylko w SR. Sieć przewiduje poprawkę, którą dodajemy do obrazu bazowego. W SR większość obliczeń odbywa się w niskiej rozdzielczości, a powiększenie jest dopiero na końcu.

Najważniejsze decyzje projektowe:

- **Uczenie poprawki (residual learning).** Sieć nie generuje całego obrazu, tylko różnicę względem obrazu bazowego: $\hat{x} = \text{baza}(y) + r_\theta(y)$. W SR bazą jest wynik bicubic, a w odszumianiu sam zaszumiony obraz. To ten sam pomysł co w VDSR i DnCNN.
- **Zerowa inicjalizacja ostatniej warstwy.** Na starcie $r_\theta(y) = 0$, czyli nieuczona sieć daje dokładnie wynik bicubic (SR) lub zwraca wejście bez zmian (odszumianie). Trening zaczyna się więc od poziomu metody bazowej i ma ją tylko poprawiać.
- **Bloki rezydualne bez BatchNorm** – jak w EDSR. Brak normalizacji nie zaburza zakresu jasności obrazu, a mnożenie wyjścia bloku przez 0,1 stabilizuje uczenie.
- **PixelShuffle na końcu (tylko SR).** Cechy liczymy dla obrazu 64×64 lub 32×32, a powiększamy je dopiero na końcu: dwa kroki ×2 dla ×4 i trzy dla ×8. Jest to dużo tańsze niż przetwarzanie obrazu 256×256 przez całą sieć.
- **Mała sieć:** $C = 24$ kanały i $B = 4$ bloki. Chcieliśmy, żeby cały eksperyment (4 modele) dało się wytrenować w kilkanaście minut na laptopie, i okazało się, że to wystarcza, żeby pokonać metody bazowe.

Tabela 4. Liczba parametrów modeli (dla porównania EDSR ma ok. 43 mln parametrów).

Wariant	Wejście	Moduł powiększania	Parametry
`sr4`	64×64	2× PixelShuffle ×2	89 859
`sr8`	32×32	3× PixelShuffle ×2	110 691
`dn001`, `dn003`	256×256	brak	48 195

5 Trening

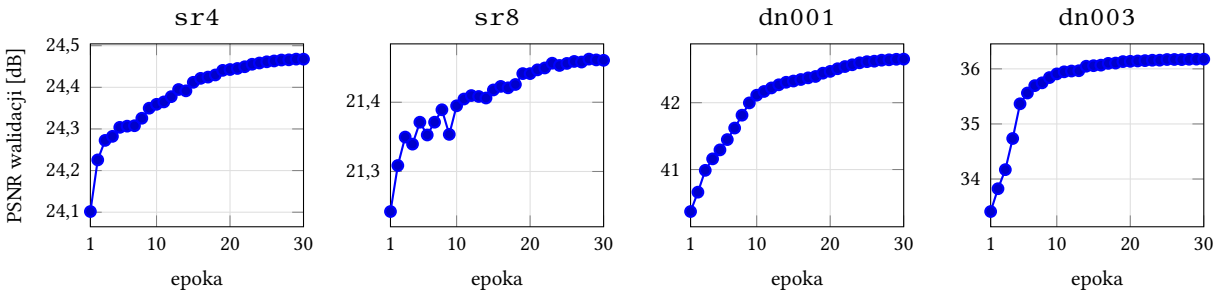
Wszystkie cztery modele trenowaliśmy z tymi samymi ustawieniami (tabela 5). Funkcją straty było MSE, bo PSNR zależy bezpośrednio od MSE. EDSR używa straty L1, ale tego wariantu nie porównywaliśmy (patrz sekcja 9). Po każdej epoce liczymy średni PSNR na zbiorze walidacyjnym i zapisujemy model z najlepszym wynikiem (`best.pt`).

Tabela 5. Ustawienia treningu (takie same dla wszystkich wariantów).

Optymalizator	Adam, lr = 10^{-3}, harmonogram kosinusowy do 10^{-5}, przycinanie gradientu do normy 1
Epoki, batch	30 epok, batch 8; każdy obraz treningowy 2 razy na epokę (1440 próbek)
Augmentacja	losowy obrót o wielokrotność 90° i odbicie poziome; w odszumianiu nowy szum w każdej epoce
Strata	MSE na obrazie RGB w zakresie $[0, 1]$, bez obcinania wyniku w trakcie treningu
Precyzja	SR: FP32; odszumianie: BF16 (autocast), wagi i optymalizator w FP32; walidacja zawsze FP32
Wybór modelu	najlepszy średni PSNR na walidacji (80 obrazów)
Sprzęt	NVIDIA RTX 5080 Laptop GPU, PyTorch 2.10.0, CUDA 12.8, ziarno 42

Tabela 6. Czas treningu i wybrane epoki.

Wariant	Czas [s]	s/epokę	Wybrana epoka	PSNR wal., ep. 1 [dB]	PSNR wal., najlepszy [dB]
`sr4`	122	4,1	29	24,10	24,47
`sr8`	168	5,6	28	21,24	21,46
`dn001`	302	10,1	30	40,37	42,65
`dn003`	303	10,1	29	33,41	36,18



Rysunek 3. PSNR na zbiorze walidacyjnym po każdej epoce (dane z `runs/main/*/history.csv`).

Na krzywych (rys. 3) widać, że modele SR uczą się szybko i od ok. 20. epoki wynik prawie się nie zmienia – zysk w całym treningu to tylko ok. 0,2–0,4 dB, ale zaczynamy już od poziomu bicubic. Modele odszumiające poprawiają się dużo bardziej (o 2,3 i 2,8 dB) i jeszcze pod koniec minimalnie rosną, więc dłuższy trening mógłby im trochę pomóc.

5.1 Eksperyment: FP32 czy BF16?

Trening odszumiania był najwolniejszy, bo sieć pracuje na obrazach 256 × 256. Sprawdziliśmy więc, czy możemy przyspieszyć go obliczeniami w BF16 bez utraty jakości. Uruchomiliśmy model `dn001` w FP32 i w BF16 z tym samym ziarnem i tym samym (CPU) generatorem szumu, a potem przenieśliśmy generowanie szumu na GPU. Wyniki pierwszych 4 epok są w tabeli 7 (logi w `runs/preliminary`).

Tabela 7. PSNR walidacji [dB] i czas epoki [s] dla `dn001` przy różnych ustawieniach precyzji.

Ustawienie	Ep. 1	Ep. 2	Ep. 3	Ep. 4	Śr. czas epoki
FP32, szum na CPU	40,346	40,599	40,854	41,064	23,2
BF16, szum na CPU	40,347	40,601	40,855	41,062	16,4
BF16, szum na GPU (końcowy)	40,372	40,661	40,988	41,160	9,7

Przy tym samym szumie FP32 i BF16 różnią się o mniej niż 0,003 dB, więc BF16 nie psuje jakości. Przeniesienie losowania szumu na GPU skróciło epokę jeszcze ok. 1,7 raza (wcześniej procesor nie

nadążał z przygotowaniem danych). Ostatni wiersz różni się trochę od pozostałych, bo generator na GPU losuje inne realizacje szumu. Te ustawienia przyjęliśmy dla końcowych modeli odszumiających.

6 Metryki

Zgodnie z instrukcją używamy wszystkich metryk z wykładu W03. Y_1 to obraz referencyjny, Y_2 wynik metody, a $N = 256 \cdot 256 \cdot 3 = 196\,608$ to liczba wartości w obrazie RGB.

- **SNE** (Squared Norm Error, niższe lepsze) – suma kwadratów różnic pikseli:

$$\mathrm{SNE} = \sum (Y_1 - Y_2)^2.$$

  Liczymy ją dla obrazów w zakresie $[0, 1]$, po wszystkich pikselach i kanałach. W skali 0–255 wartości byłyby 255^2 razy większe.

- **PSNR** (wyższe lepsze):

$$\mathrm{PSNR} = 10 \log_{10} \frac{\mathrm{MAX}^2}{\mathrm{MSE}}, \qquad \mathrm{MSE} = \frac{1}{N} \sum (Y_1 - Y_2)^2, \qquad \mathrm{MAX} = 1.$$

- **SSIM** (wyższe lepsze) – funkcja `structural_similarity` z modułu `skimage.metrics`, okno gaussowskie $\sigma = 1{,}5$, `data_range=1`, liczone dla każdego kanału RGB i uśrednione.
- **LPIPS** (niższe lepsze) – biblioteka `lpips` 0.1.4, wersja 0.1, sieć SqueezeNet z wytrenowanymi wagami, obrazy przeskalowane do $[-1, 1]$.

Metryki liczymy dla każdego obrazu osobno, na pełnym obrazie 256×256 (bez obcinania brzegów), na wartościach `float` jeszcze przed zapisem do PNG. W tabelach podajemy średnią z wszystkich 100 obrazów testowych. Warto zauważyć, że dla jednego obrazu SNE i PSNR niosą tę samą informację ($\mathrm{PSNR} = 10 \log_{10}(N/\mathrm{SNE})$), ale po uśrednieniu po obrazach już nie: na średnie SNE mocno wpływają najtrudniejsze obrazy z dużym błędem. Wartości LPIPS dla SqueezeNet nie należy porównywać wprost z artykułami, w których używano AlexNet.

7 Wyniki

Wszystkie wyniki pochodzą z pełnej ewaluacji na 100 obrazach testowych. Średnie są zapisane w pliku `runs/evaluation/summary.csv`, a wyniki każdego obrazu w `per_image.csv` w tym samym katalogu. Najlepszy wynik w każdym wariancie jest pogrubiony.

7.1 Zwiększanie rozdzielczości

Tabela 8. Zwiększanie rozdzielczości do 256×256, zbiór testowy DIV2K (100 obrazów). Wejście zmniejszone przez `cv2.resize` z `INTER_AREA`. ↑ – wyższe lepsze, ↓ – niższe lepsze.

Metoda	PSNR [dB] ↑	SSIM ↑	LPIPS ↓	SNE ↓
bicubic OpenCV, SR ×4 ($64 \times 64 \to 256 \times 256$)	22,91	0,6445	0,3978	1 216,2
ResidualNet, SR ×4 ($64 \times 64 \to 256 \times 256$)	**23.63**	**0.6898**	**0.2886**	**1055.0**
bicubic OpenCV, SR ×8 ($32 \times 32 \to 256 \times 256$)	20,30	0,4673	0,7355	2 148,5
ResidualNet, SR ×8 ($32 \times 32 \to 256 \times 256$)	**20.75**	**0.5001**	**0.5990**	**1962.0**

7.2 Odszumianie

Tabela 9. Odszumianie obrazów 256 × 256 z szumem gaussowskim, zbiór testowy DIV2K (100 obrazów). Dla porównania dodaliśmy wiersz z samym zaszumionym wejściem (bez żadnej filtracji).

Metoda	PSNR [dB] ↑	SSIM ↑	LPIPS ↓	SNE ↓
wejście zaszumione, szum gaussowski $\sigma = 0{,}01$	40,07	0,9709	0,0179	19,4
denoise bilateral, szum gaussowski $\sigma = 0{,}01$	38,79	0,9816	0,0143	27,5
ResidualNet, szum gaussowski $\sigma = 0{,}01$	**42.56**	**0.9881**	**0.0115**	**11.1**
wejście zaszumione, szum gaussowski $\sigma = 0{,}03$	30,60	0,8285	0,1009	171,6
denoise bilateral, szum gaussowski $\sigma = 0{,}03$	31,35	0,9192	0,0608	152,8
ResidualNet, szum gaussowski $\sigma = 0{,}03$	**36.01**	**0.9664**	**0.0221**	**52.2**

7.3 Wyniki dla pojedynczych obrazów

Same średnie mogą ukrywać to, że na części obrazów sieć wypada gorzej. Dlatego policzyliśmy, na ilu obrazach testowych sieć jest lepsza od metody bazowej w każdej metryce, oraz jak rozkłada się różnica PSNR (tabela 10).

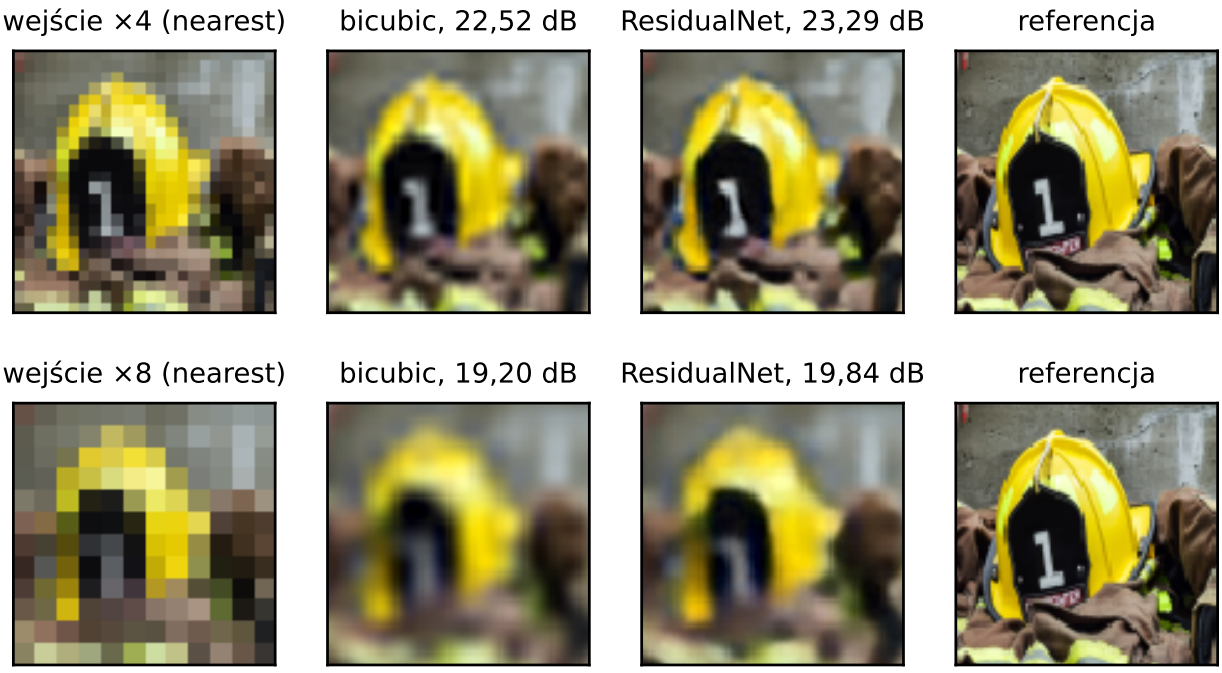
Tabela 10. Porównanie ResidualNet z metodą bazową na pojedynczych obrazach testowych (na 100). ΔPSNR = PSNR sieci − PSNR metody bazowej.

	Sieć lepsza na … obrazach				ΔPSNR [dB]		
Wariant	PSNR	SSIM	LPIPS	SNE	mediana	min.	maks.
SR ×4	100	100	100	100	0,57	0,10	2,94
SR ×8	100	100	100	100	0,36	0,05	2,39
szum $\sigma = 0{,}01$	99	87	78	99	3,90	−0,73	5,98
szum $\sigma = 0{,}03$	100	100	95	100	4,60	1,88	7,55

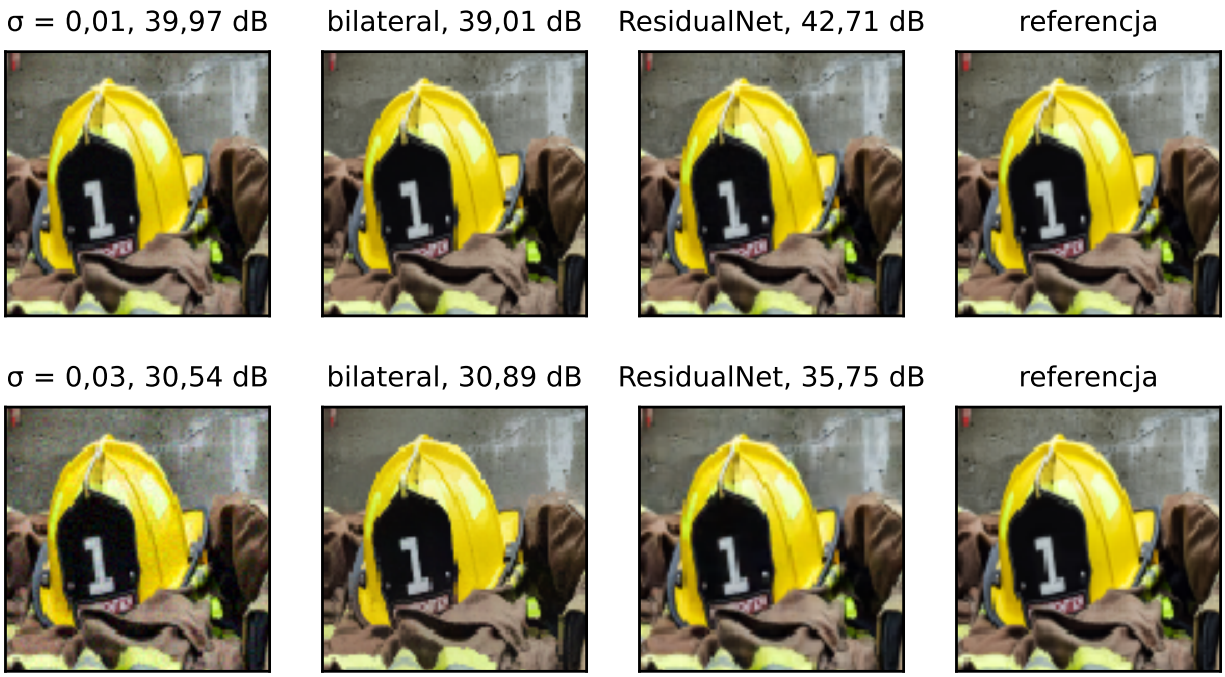
W SR sieć jest lepsza na każdym obrazie testowym, chociaż na niektórych zysk jest bardzo mały (0,05 dB). Przy słabym szumie ($\sigma = 0{,}01$) jest jeden obraz (0868), na którym filtr bilateralny dał wyższy PSNR, a w SSIM i LPIPS sieć przegrywa odpowiednio na 13 i 22 obrazach. Przy tak słabym szumie filtr bilateralny prawie nic nie zmienia na jednolitych fragmentach, a sieć czasem lekko wygładza drobne tekstury – PSNR to wybacza, ale metryki strukturalne i percepcyjne już nie.

7.4 Wyniki wizualne

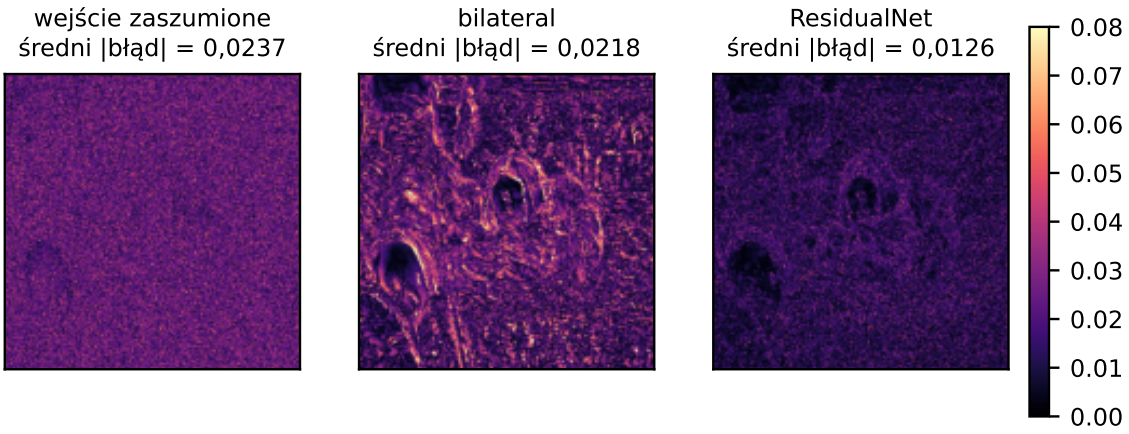
Rysunki 4–6 pokazują ten sam fragment obrazu 0721 (z naszej walidacji) dla wszystkich metod. Wartości PSNR w tytułach dotyczą całego obrazu zapisanego jako 8-bitowy PNG, więc mogą się trochę różnić od wartości liczonych na danych `float`. Porównania z mapami błędu dla obrazów testowych są w dodatku A i w raporcie `runs/evaluation/raport.html`.



Rysunek 4. Zwiększanie rozdzielczości, wycinek 96 × 96. Sieć daje wyraźniejsze krawędzie kasku i cyfry „1” niż bicubic. Przy ×8 obu metodom brakuje szczegółów – z obrazu 32 × 32 nie da się ich odtworzyć, a sieć uczona na MSE woli bezpieczne, lekko rozmyte przewidywania.



Rysunek 5. Odszumianie, wycinek 96 × 96. Przy $\sigma = 0{,}01$ szum jest prawie niewidoczny, a filtr bilateralny obniża PSNR względem samego wejścia, bo rusza też szczegóły obrazu. Przy $\sigma = 0{,}03$ sieć usuwa ziarno na kasku i tle, zachowując ostre krawędzie.



Rysunek 6. Mapy średniego błędu bezwzględnego RGB dla $\sigma = 0{,}03$ (cały obraz). Błąd zaszumionego wejścia jest równomierny. Filtr bilateralny zmniejsza go na gładkich powierzchniach, ale tworzy duże błędy na teksturach (materiał kurtki, beton). Błąd sieci jest najmniejszy i skupia się głównie na krawędziach.

8 Wnioski

1. **Mała sieć wystarczyła, żeby pokonać metody bazowe.** ResidualNet ma 48–111 tys. parametrów i trenuje się od 2 do 5 minut na model, a mimo to we wszystkich czterech wariantach jest lepsza od bicubic i od dostrojonego filtru bilateralnego w każdej z metryk SNE, PSNR, SSIM i LPIPS.
2. **Odszumianie zyskało dużo więcej niż SR** (+3,8 / +4,7 dB wobec +0,7 / +0,5 dB). Szum jest losowy i niezależny od treści obrazu, więc sieć łatwo go odróżnia od krawędzi i tekstur. W SR informacja usunięta przy zmniejszaniu jest tracona na stałe – to problem odwrotny z wykładu W04, w którym wiele obrazów HR daje ten sam obraz LR. Przy ×8 tracimy 98% pikseli, więc zysk jest najmniejszy.
3. **W SR LPIPS poprawił się bardziej, niż sugeruje PSNR.** Przy ×4 LPIPS spadł z 0,398 do 0,289 (o 27%), a PSNR wzrósł tylko o 0,7 dB. Obrazy wyglądają więc wyraźnie lepiej, mimo niewielkiej poprawy w metrykach pikselowych.
4. **Metryki nie zawsze się zgadzają.** Filtr bilateralny przy $\sigma = 0{,}01$ obniża PSNR względem samego wejścia (40,07 → 38,79 dB), ale poprawia SSIM i LPIPS. Dla tego samego wariantu sieć wygrywa w PSNR na 99 obrazach, a w LPIPS tylko na 78. Pokazuje to kompromis między dokładnością pikselową a jakością percepcyjną (perception–distortion trade-off), dlatego warto patrzeć na kilka metryk naraz.
5. **Metoda klasyczna jest bardzo wrażliwa na parametry.** Wybór `sigma_color` zmieniał wynik filtru bilateralnego nawet o 7 dB. Bez strojenia przewaga sieci byłaby dużo większa, ale wtedy porównanie nie byłoby uczciwe.
6. **BF16 przyspiesza trening bez utraty jakości.** Razem z generowaniem szumu na GPU trening odszumiania był ok. 2,4 raza szybszy niż w FP32 z szumem na CPU.

9 Ograniczenia i co można by zrobić dalej

- **Zbiór testowy.** Używamy oficjalnej walidacji DIV2K (0801–0900), bo referencje oficjalnego testu nie są publiczne. Kod pozwala podmienić zbiór testowy (`prepare --test-dir`), gdyby udało się go zdobyć.
- **Jeden trening na wariant.** Każdy model trenowaliśmy raz, z jednym ziarnem, więc nie wiemy, jak bardzo wyniki zależą od losowości. Przy SR ×8, gdzie zysk wynosi 0,45 dB, warto by to sprawdzić.
- **Mała sieć i krótki trening.** Modele odszumiające jeszcze się poprawiały w 30. epoce. Następnym krokiem byłaby większa sieć (np. 32 kanały, 6 bloków) i 60 epok – kod już to obsługuje.

- **Inna funkcja straty.** EDSR uczy się z L1, a SRGAN/ESRGAN ze stratą percepcyjną. Strata L1 (`train --loss l1`) mogłaby poprawić LPIPS kosztem PSNR.
- **Proporcje obrazu.** Cały obraz skalujemy do kwadratu, więc obiekty są lekko zniekształcone. Alternatywą byłoby wycinanie kwadratowego fragmentu, ale wtedy część obrazu nie trafiłaby do testu.

10 Jak uruchomić

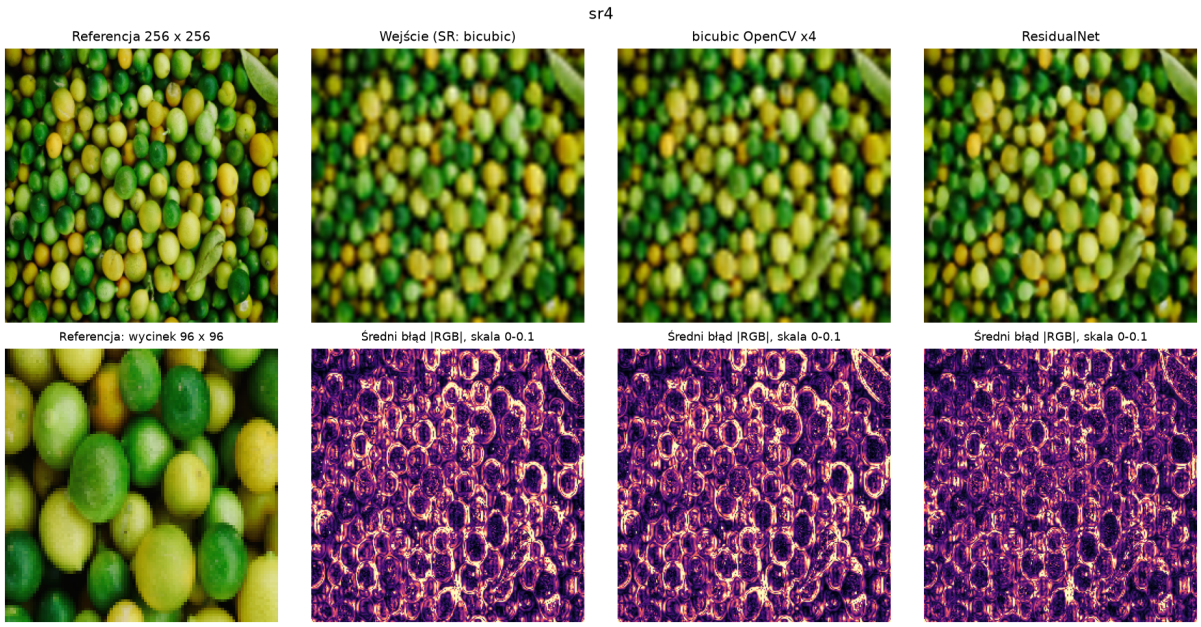
Instalacja: `python -m pip install -r requirements.txt` (Python ≥ 3.10). Najważniejsze komendy:

`python -m sigk download`, `prepare`, `verify`	pobranie DIV2K, przygotowanie 256 × 256, kontrola danych
`python -m sigk train --condition sr4`	trening jednego wariantu (`sr4`, `sr8`, `dn001`, `dn003`)
`python -m sigk tune-baseline`	strojenie filtru bilateralnego na walidacji
`python -m sigk evaluate --sne-definition sse ...`	pełna ewaluacja i raport HTML
`python -m sigk predict --checkpoint ... --input ...`	przetworzenie własnego obrazu

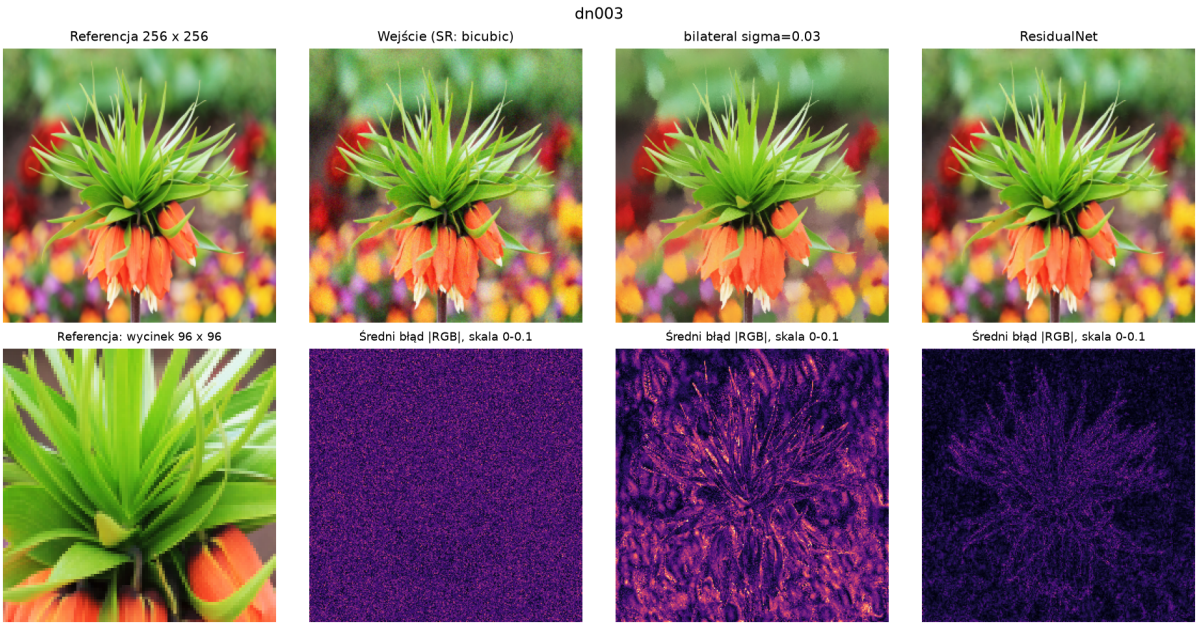
Pełny opis jest w `README.md`, a cały eksperyment odtwarza skrypt `scripts/run_all.ps1`. Rysunki do tego sprawozdania generuje `docs/sprawozdanie/make_figures.py`.

A Przykłady z raportu dla obrazów testowych

Poniższe rysunki pochodzą z raportu wygenerowanego podczas ewaluacji. Górny wiersz: referencja, wejście, metoda bazowa i ResidualNet. Dolny wiersz: powiększony wycinek referencji i mapy średniego błędu RGB (skala 0–0,1). W SR kolumna „wejście” pokazuje obraz już powiększony metodą bicubic, więc jest taka sama jak kolumna metody bazowej. Wszystkie 20 porównań (po 5 na wariant) jest w `runs/evaluation/images`.



Rysunek 7. SR ×4, obraz testowy 0802 (PSNR: bicubic 24,39 dB, ResidualNet 25,69 dB).



Rysunek 8. Odszumianie $\sigma = 0{,}03$, obraz testowy 0803 (PSNR: bilateral 29,24 dB, ResidualNet 36,33 dB).